

LABORATOR 4

MARIA PREDA

1. BREVIAR TEORETIC

1.1. **Construcția automatelor pentru operații binare cu rezultat DFA.** Fie

$$A_1 = (Q_1 = \{q_0, q_1, q_2, \dots\}, \Sigma_1, q_0, F_1, \delta_1)$$

$$A_2 = (Q_2 = \{r_0, r_1, r_2, \dots\}, \Sigma_2, r_0, F_2, \delta_2)$$

două **DFA-uri complet definite**. Vrem să construim automatul

$$A = (Q, \Sigma, (q_0, r_0), F, \delta)$$

pentru fiecare din operațiile:

- **Reuniune:**

- Stările: $Q = Q_1 \times Q_2$
- Funcția de tranziție: $\delta((q_i, r_j), x) = (\delta_1(q_i, x), \delta_2(r_j, x)), \forall (q_i, r_j) \in Q, \forall x \in \Sigma$
- Starea inițială: (q_0, r_0)
- Stările finale: $F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

- **Intersecție:**

- Stările: $Q = Q_1 \times Q_2$
- Funcția de tranziție: $\delta((q_i, r_j), x) = (\delta_1(q_i, x), \delta_2(r_j, x)), \forall (q_i, r_j) \in Q, \forall x \in \Sigma$
- Starea inițială: (q_0, r_0)
- Stările finale: $F = F_1 \times F_2$

- **Diferență:**

- Stările: $Q = Q_1 \times Q_2$
- Funcția de tranziție: $\delta((q_i, r_j), x) = (\delta_1(q_i, x), \delta_2(r_j, x)), \forall (q_i, r_j) \in Q, \forall x \in \Sigma$
- Starea inițială: (q_0, r_0)
- Stările finale: $F = F_1 \times (Q_2 \setminus F_2)$

1.2. **Algoritmul de transformare a unui NFA în DFA.** Știm că principala diferență între un NFA și un DFA constă în faptul că NFA-ul admite tranziții, dintr-o anumită stare, cu un același simbol, către mai mult de o singură stare. Astfel, o idee intuitivă ar fi, ca de fiecare dată când putem ajunge dintr-o stare cu un simbol în mai multe stări, să transformăm toate aceste stări destinație într-una singură.

Observație 1.1. Stările noului DFA creat vor fi submulțimi ale mulțimii stărilor NFA-ului, starea inițială rămânând aceeași. Funcția de tranziție devine:

$$\delta'(R, x) = \bigcup_{q \in R} \delta(q, x), \forall R \in P(Q), \forall x \in \Sigma$$

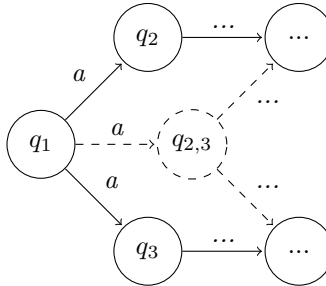
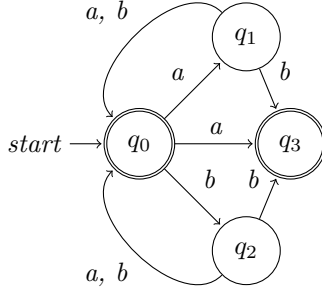


FIGURA 1. Stări DFA obținute din unirea mai multor stări NFA

Observație 1.2. Stările finale ale DFA-ului nou construit vor fi toate stările care, în componența lor, conțin cel puțin o stare care era înainte finală.

$$F' = \{R \mid R \in Q', R \cap F \neq \emptyset\}$$

Exemplu 1.3. Vom transforma urmatorul NFA într-un DFA:



Vom face întâi tabelul pentru NFA, aratând pentru fiecare pereche (stare, simbol) în ce stări se poate ajunge.

Stare	a	b
q0	$q_{13} = \{q_1, q_3\}$	q2
q1	q0	$q_{03} = \{q_0, q_3\}$
q2	q0	$q_{03} = \{q_0, q_3\}$
q3	$\emptyset$	$\emptyset$

Vom construi un tabel de tranziții pentru noul DFA în felul următor: pe prima linie punem starea initială și stările în care se poate ajunge din ea cu fiecare simbol. Apoi, pe următoarea linie punem prima nouă stare în care apare în tabel și tot așa, până când tuturor stărilor în care se poate ajunge le corespunde o linie.

Stare	a	b
q0	$q_{13} = \{q_1, q_3\}$	q2
q13	q0	$q_{03} = \{q_0, q_3\}$
q2	q0	$q_{03} = \{q_0, q_3\}$
q03	$q_{13} = \{q_1, q_3\}$	q2

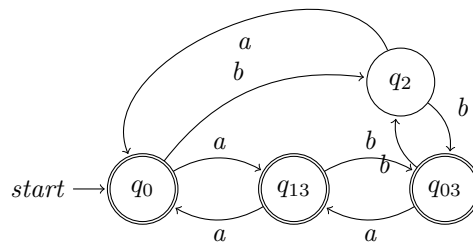


FIGURA 2. DFA-ul rezultat

1.3. Algoritmul de transformare a unui λ -NFA în NFA. Încercăm să eliminăm λ -tranzițiile. Deci, în automatul rămas, dintr-o stare în alta vom putea trece doar prin caractere din Σ diferite de λ .

Asa cum am văzut și în primul laborator, mulțimea de stări în care putem ajunge dintr-o stare cu un anumit caracter se numește λ -inchiderea stării și se notează cu $\langle \text{stare} \rangle$.

Observație 1.4. Orice stare face parte din propria λ -inchidere.

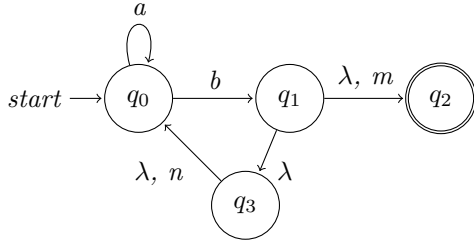
$$\langle q \rangle = \{r \mid r \in \hat{\delta}(q, \lambda^*)\}, \text{ unde } \hat{\delta} \text{ reprezintă compunerea mai multor tranziții succesive}$$

Idee: Vom construi un nou automat astfel încât aflându-ne la o stare q , tranzițiile pentru un simbol c se duc înspre stările din λ -NFA în care puteam să ajungem cu $\lambda^*c\lambda^*$.

Observație 1.5. Starea inițială rămâne aceeași.

Observație 1.6. Stările finale sunt acele stări care au în λ -închidere cel puțin o stare finală.

Exemplu 1.7. Fie următorul automat λ -NFA:



Pas 1: Determinăm λ -închiderile:

$$\begin{aligned}\langle q_0 \rangle &= \{q_0\} \\ \langle q_1 \rangle &= \{q_1, q_2, q_3, q_0\} \\ \langle q_2 \rangle &= \{q_2\} \\ \langle q_3 \rangle &= \{q_3, q_0\}\end{aligned}$$

Stările finale în NFA vor fi $F' = \{q_1, q_2\}$, deoarece $\langle q_1 \rangle \cap F = \{q_2\} \neq \emptyset$ și $\langle q_2 \rangle \cap F = \{q_2\} \neq \emptyset$ (ambele îl conțin pe q_2 în λ -închidere, care este stare finală).

Pas 2: Realizăm tabelul de tranziții:

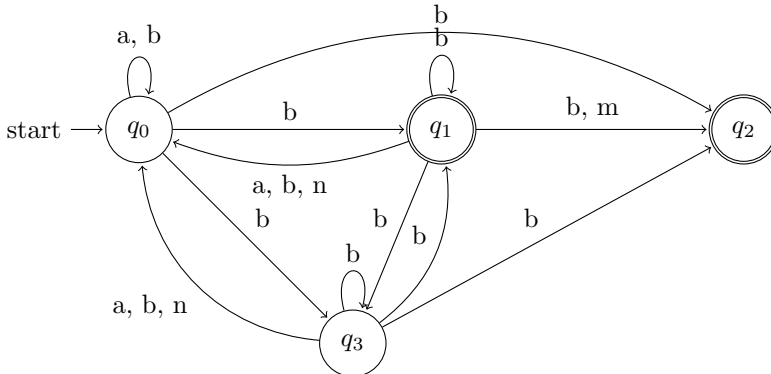
stare	a	b	m	n	λ	λ^*
q_0	$\{q_0\}$	$\{q_1\}$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_0\}$
q_1	$\emptyset$	$\emptyset$	$\{q_2\}$	$\emptyset$	$\{q_2, q_3\}$	$\{q_1, q_2, q_3, q_0\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_2\}$
q_3	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_0\}$	$\{q_0\}$	$\{q_0, q_3\}$

Pas 3: Determinăm tranzițiile de forma $\lambda^* c \lambda^*$:

stare	$\lambda^* a \lambda^*$	$\lambda^* b \lambda^*$	$\lambda^* m \lambda^*$	$\lambda^* n \lambda^*$
q_0	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\emptyset$	$\emptyset$
q_1	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_2\}$	$\{q_0\}$
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_3	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\emptyset$	$\{q_0\}$

Pas 4: Realizăm tabelul de tranziții pentru NFA, și reprezentăm automatul:

stare	a	b	m	n
q_0 (starea inițială)	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\emptyset$	$\emptyset$
$q_1 \in F$	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\{q_2\}$	$\{q_0\}$
$q_2 \in F$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
q_3	$\{q_0\}$	$\{q_0, q_1, q_2, q_3\}$	$\emptyset$	$\{q_0\}$



1.4. Algoritmi de minimizare a unui DFA. De multe ori, automatul determinist obtinut in urma unei constructii are mai multe stari decat este necesar. In acest caz, putem construi un DFA echivalent, cu un numar minim de stari.

Definiție 1.8. *Un DFA minim este un DFA echivalent cu automatul initial, care accepta acelasi limbaj si are numarul minim posibil de stari.*

Ideea generala a minimizarii este sa identificam starile care se comporta la fel pentru orice cuvânt citit in continuare. Aceste stari se vor numi echivalente si vor putea fi unite.

Definiție 1.9. *Doua stari $p, q \in Q$ se numesc echivalente daca, pentru orice cuvânt $w \in \Sigma^*$, automatul ajunge fie din ambele stari intr-o stare finala, fie din ambele stari intr-o stare nefinala dupa citirea lui w .*

Observație 1.10. *Daca exista un cuvânt w care, citit din doua stari, duce dintr-una intr-o stare finala si din cealalta intr-o stare nefinala, atunci cele doua stari nu sunt echivalente si nu pot fi unite.*

Vom prezenta in continuare doi algoritmi clasici de minimizare: algoritmul lui Moore si algoritmul lui Hopcroft. Ambele pornesc de la aceeasi idee: impartim multimea starilor in clase de echivalenta pana cand aceste clase nu mai pot fi separate.

1.4.1. Algoritmul lui Moore. Idee: Vom porni de la o impartire foarte simpla a starilor, in finale si nefinale. Apoi, vom desparti in mod repetat starile care, desi se afla momentan in aceeasi clasa, se comporta diferit la citirea simbolurilor din alfabet.

Observație 1.11. *La fiecare pas, o stare este determinata de clasa din care face parte si de clasele in care ajunge prin fiecare simbol din alfabet.*

Astfel, doua stari vor ramane in aceeasi clasa doar daca: sunt ambele finale sau ambele nefinale, iar pentru fiecare simbol $a \in \Sigma$, prin tranzitia cu a ajung in stari aflate in aceleasi clase.

Pasii algoritmului sunt urmatoarii:

Pas 1: Eliminam starile inaccesibile.

Pas 2: Construim partitia initiala a multimii starilor:

$$P_0 = \{F, Q \setminus F\}$$

adica punem separat starile finale si starile nefinale.

Pas 3: Pentru fiecare clasa din partitita curenta, verificam daca toate starile din ea au acelasi comportament fata de clasele in care ajung cu fiecare simbol din alfabet.

Mai precis, pentru o stare q , asociem semnatura:

$$([\delta(q, a_1)], [\delta(q, a_2)], \dots, [\delta(q, a_k)]),$$

unde $[x]$ reprezinta clasa in care se afla starea x , iar $\Sigma = \{a_1, a_2, \dots, a_k\}$.

Pas 4: Daca intr-o aceeasi clasa exista stari cu semnaturi diferite, atunci acea clasa se desparte in mai multe clase, cate una pentru fiecare semnatura distincta.

Pas 5: Repetam pasul anterior pana cand partitia nu se mai modifica.

Pas 6: Clasele obtinute la final sunt clasele de echivalenta ale starilor. Fiecare clasa va deveni o singura stare in DFA-ul minim.

Observație 1.12. *Starile finale ale noului automat vor fi clase de echivalenta care contin cel puțin o stare finala.*

Observație 1.13. *Starea initiala a noului automat este clasa de echivalenta care contine vechea stare initiala.*

1.4.2. Algoritmul lui Hopcroft. Idee: Si in acest algoritm pornim de la impartirea in stari finale si nefinale, dar separarea finala se face mai eficient. In loc sa verificam de fiecare data toate clasele in intregime, pastram o multime de clase care trebuie folosite pentru a separa alte stari.

Observație 1.14. *Algoritmul lui Hopcroft este mai eficient decat algoritmul lui Moore si este preferat in implementari pentru automate mari.*

Intuitiv, dacă o clasă C a fost deja identificată, atunci ne putem întreba ce stări ajung în C printr-un anumit simbol. Dacă într-o clasă există unele stări care ajung în C și altele care nu ajung în C , atunci acea clasă trebuie despartită.

Vom nota cu:

$$\text{Pre}_a(C) = \{q \in Q \mid \delta(q, a) \in C\}$$

multimea stărilor care ajung în clasa C prin simbolul a .

Pasii algoritmului sunt următorii:

Pas 1: Eliminăm stările inaccesibile.

Pas 2: Construim partiția inițială:

$$P = \{F, Q \setminus F\}.$$

Pas 3: Inițializăm o mulțime de lucru W , care va conține clasele ce vor fi folosite pentru separare. De obicei, începem cu una dintre clasele partiției inițiale sau cu ambele.

Pas 4: Cât timp W nu este vidă, alegem o clasă $C \in W$ și o eliminăm din mulțimea de lucru.

Pas 5: Pentru fiecare simbol $a \in \Sigma$, determinăm mulțimea:

$$\text{Pre}_a(C) = \{q \in Q \mid \delta(q, a) \in C\}.$$

Pas 6: Pentru fiecare clasă R din partiția curentă, verificăm dacă ea se intersectează nevid cu $\text{Pre}_a(C)$, dar nu este inclusă complet în această mulțime.

În acest caz, clasa R trebuie împartită în două subclase:

$$R_1 = R \cap \text{Pre}_a(C), \quad R_2 = R \setminus \text{Pre}_a(C).$$

Pas 7: Înlocuim clasa R din partiție cu cele două subclase R_1 și R_2 .

Pas 8: Actualizăm și mulțimea de lucru W :

- (1) dacă R se afla deja în W , atunci o înlocuim cu R_1 și R_2 ;
- (2) altfel, o adăugăm în W pe cea mai mică dintre cele două subclase, pentru a păstra eficiența algoritmului.

Pas 9: Repetăm procesul până când nu se mai poate face nicio despartire. Clasele obținute la final sunt clasele de echivalență și vor reprezenta stările DFA-ului minim.

Observație 1.15. *Si în acest caz, stările finale ale noului automat sunt clasele care conțin cel puțin o stare finală, iar starea inițială este clasa care conține vechea stare inițială.*

Observație 1.16. *DFA-ul minim echivalent unui limbaj regulat este unic (nu considerăm redenumirea stărilor ca fiind un automat diferit).*

Observație 1.17. *Minimizarea este utilă mai ales înainte de transformarea unui automat în expresie regulată, deoarece un număr mai mic de stări va duce, în general, la calcule mai simple și la o expresie regulată finală mai ușor de obținut.*

2. EXPRESII REGULATE

Notație 2.1. *Familia expresiilor regulate se notează $RegEx$.*

Observație 2.2. *Avem următoarele operații posibile peste $RegEx$, care seamănă cu operațiile pentru cuvinte și limbaje pe care le-am făcut până acum:*

- $"+" = \text{echivalentul reuniunii}$
- $"." = \text{echivalentul concatenării}$
- $"*" = \text{stelarea}$
- $"()" = \text{care se comportă ca parantezele normale}$

Definiție 2.3. *Expresiile regulate peste Σ fac parte din familia expresiilor regulate. Această familie se definește recursiv în felul următor:*

- (1) $\emptyset, \lambda \in RegEx$ și $a \in RegEx, \forall a \in \Sigma$ (cazul de bază)

$$(2) \quad e_1, e_2 \in RegEx \Rightarrow (e_1 + e_2) \in RegEx$$

$$(3) \quad e_1, e_2 \in RegEx \Rightarrow (e_1 \cdot e_2) \in RegEx$$

$$(4) \quad e \in RegEx \Rightarrow e^* \in RegEx$$

Observație 2.4. *Ca de fiecare data cand sunt definiti operatori, avem nevoie si de o prioritate a lor, numita precedenta. Ordinea acestora, de la precedenta cea mai mare la cea mai mica, este:*

$$"()" > "*" > "." > "+"$$

De multe ori, urmarirea unui limbaj (definit ca pana acum) sau a unui automat care accepta un limbaj este complicata, mai ales in cazurile in care automatul are foarte multe stari.

Observație 2.5. *Putem simplifica automatele si limbajele cu ajutorul expresiilor regulate.*

Observație 2.6. *Pentru orice automat finit exista o expresie regulata echivalenta.*

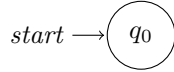
Observație 2.7. *Pentru orice expresie regulata exista un automat finit care o accepta.*

Scopul acestui laborator este sa vedem cum putem transforma expresii regulate in automate si automate in expresii regulate.

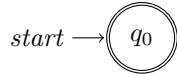
Vom porni de la cazurile de baza, prezente in Definitia 1.3.

(1) *Avem cele trei situatii de expresii regulate:*

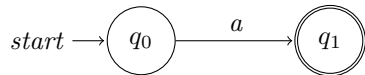
- $e = \emptyset \Rightarrow L = \emptyset$



- $e = \lambda \Rightarrow L = \{\lambda\}$

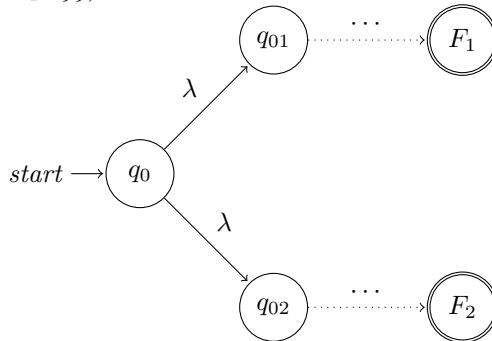


- $e = a, a \in \Sigma \Rightarrow L = \{a\}$



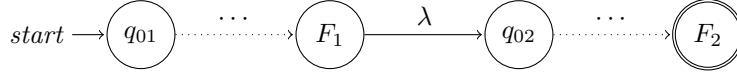
(2) $e = e_1 + e_2 \Rightarrow L(e) = L(e_1) \cup L(e_2)$

Automatul finit pentru $L(e_1 + e_2)$ este $(Q_1 \cup Q_2 \cup \{q_0\}, \Sigma_1 \cup \Sigma_2, q_0, F_1 \cup F_2, \delta_1 \cup \delta_2 \cup \{\delta(q_0, \lambda) = \{q_{01}, q_{02}\}\})$



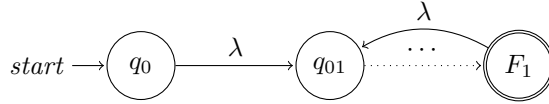
$$(3) \quad e = e_1 \cdot e_2 \Rightarrow L(e) = L(e_1) \cdot L(e_2)$$

Automatul finit pentru $L(e_1 \cdot e_2)$ este $(Q_1 \cup Q_2, \Sigma_1 \cup \Sigma_2, q_{01}, F_2, \delta_1 \cup \delta_2 \cup \{\delta(q_{f_1}, \lambda) \ni q_{02} \mid \forall f_1 \in F_1\})$



$$(4) \quad e = (e_1)^* \Rightarrow L(e) = (L(e_1))^*$$

Automatul finit pentru $(L(e_1))^*$ este $(Q_1 \cup \{q_0\}, \Sigma_1, q_0, F_1 \cup \{q_0\}, \delta_1 \cup \{\delta(q_0, \lambda) = q_{01}\} \cup \{\delta(q_{f_1}, \lambda) \ni q_{01} \mid \forall f_1 \in F_1\})$



Definiție 2.8. Se numeste automat finit extins (AFE) un automat in care, spre deosebire de cele obisnuite, functia de tranzitie este inlocuita de o functie de etichetare $et : Q \times Q \rightarrow RegEx(\Sigma)$.

2.1. Algoritm de transformare λ -NFA intr-o expresie regulata. Intuitie: Automatul finit extins are pe muchii expresii regulate. Atunci, am putea transforma intai automatul finit initial intr-unul extins, pentru ca ulterior, eliminand stare cu stare, sa ajungem cu doar doua stari, cea initiala si cea finala, intre care avem o expresie regulata.

In continuare, vom prezenta pasii algoritmului:

Pas 1: Daca intre doua stari exista mai mult de o singura tranzitie, le vom comprima intr-o singura expresie regulata.

$$et(q_x, q_y) = \{w \in REX(\Sigma) \mid w = a_1 + a_2 + \dots + a_n; q_y \in \delta(q_x, a_i), a_i \in (\Sigma \cup \{\lambda\}), \forall i \in \{1, \dots, n\}\}$$

Pas 2: Se va adauga o noua stare initiala daca se intampla cel putin unul dintre cele doua cazuri:

- Starea initiala este si finala.
- Exista tranzitii care vin catre starea initiala dinspre alte stari sau bucle.

Noua stare initiala va pleca spre o singura stare cu λ , aceasta starea fiind vechea stare initiala.

Pasul 3: Trebuie sa avem o singura stare finala, din care nu mai putem pleca. Daca aceste conditii nu sunt indeplinite, vom adauga o noua stare finala, spre care vom avea λ -tranzitie din toate vechile stari finale (acestea nu mai sunt finale acum).

Pasul 4: Se elimina pe rand, una cate una, toate starile, pana cand ajungem sa mai avem doar starea initiala si starea finala.

Observație 2.9. Ordinea in care alegem sa eliminam starile nu va afecta rezultatul. De obicei se elimină starea cu cele mai puține săgeți.

Distingem urmatoarele cazuri la eliminarea unei stari:

- Pentru fiecare pereche de tranzitii (una care intra in starea pe care o eliminam si una care pleaca din ea), vom pune o noua tranzitie intre starea din care ajungeam si cea spre care ne duceam, pe care vom pune expresia regulata obtinuta prin concatenarea expresiilor regulate de pe muchiile vechi. Daca mai exista o tranzitie între aceste doua stari, vom reuni expresiile regulate.

Pasul 5: Cand raman doar starea initiala si cea finala, expresia regulata de pe tranzitia intre ele este cea echivalenta limbajului acceptat de automat.

Observație 2.10. La **pasul 4** se poate elimina orice stare, prin unirea oricărui predecesor cu oricare succesor al acesteia.

Observație 2.11. Reuniunea se include între paranteze, pentru a se executa înainte de concatenare și se încearcă simplificarea cât mai mult a expresiilor rezultate.

3. CLASA LIMBAJELOR INDEPENDENTE DE CONTEXT

În cursurile/seminarele/laboratoarele anterioare am observat faptul că există anumite limbaje care nu se pot modela prin intermediul automatelor finite adică nu erau limbaje regulate.

Exemplu 3.1. Următoarele limbaje nu sunt regulate:

- $L_1 = \{a^n b^n \mid n \geq 0\}$
- $L_2 = \{ww^R \mid w \in \Sigma^*\}$

O parte dintre limbajele care nu fac parte din clasa limbajelor regulate fac parte dintr-o clasă numită **limbaje independente de context**. Cele două limbaje din Exemplul 3.1 vor face parte din această familie.

Teoremă 3.2. Fie CFL familia limbajelor independente de context. Atunci:

$$\text{REG} \subset \text{CFL}$$

Observație 3.3. Există mai multe familii de limbaje definite conform ierarhiei lui Chomsky:

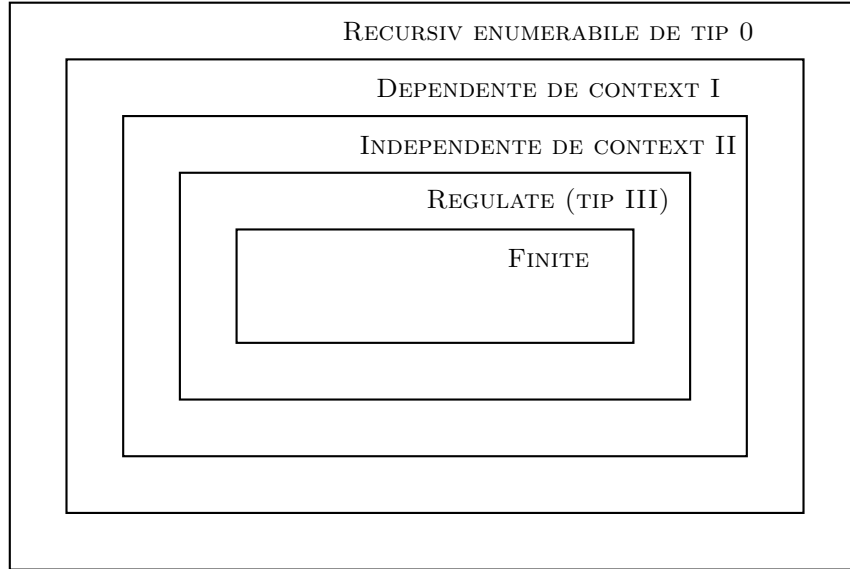


FIGURA 3. Ierarhia lui Chomsky. Sursa: A. Păun 2023, Curs 12: Decidabilitate. Limbaje Formale și Automate

Pentru a defini clasa limbajelor regulate ne-am folosit de automate finite, considerând că un limbaj este regulat dacă și numai dacă se poate construi un automat care să accepte toate cuvintele din acesta, și numai pe acestea. Vom încerca să definim astfel și clasa limbajelor independente de context.

Definiție 3.4. Un limbaj independent de context este un limbaj acceptat de un automat push-down.

Această definiție este într-o oarecare măsură circulară, pe viitor vom da o definiție mai riguroasă, utilizând gramatici.

3.1. Automate push-down (PDA). Spre deosebire de automatele cu care am lucrat până acum, automatele push-down vin în plus cu o stivă de caractere, în care se scot și se adaugă anumite simboluri auxiliare la fiecare tranziție.

Definiție 3.5. Un automat push-down (PDA) este un tuplu $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ cu următoarea semnificație:

- Q : Mulțimea de stări
- Σ : Alfabetul de input
- Γ : Alfabetul stivei

- $\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \times Q \times \Gamma^*$, relație de tranziție.
- $q_0 \in Q$: starea inițială.
- $Z_0 \in \Gamma$: Simbolul de la capătul stivei.
- $F \subset Q$: mulțimea stărilor finale.

Sistemul de tranziție: Fie $(s, \sigma, \gamma, t, \tau) \in \delta$ o tranziție într-un automat push-down. Pornim dintr-o stare $s \in Q$ și vrem să consumăm din cuvânt un simbol $\sigma \in \Sigma \cup \{\lambda\}$, doar dacă la vârful stivei se regăsește simbolul auxiliar γ , astfel ajungând la starea t , după ce am scos de la vârful stivei simbolul γ (pop) și am adău gat (push) simbolul τ .

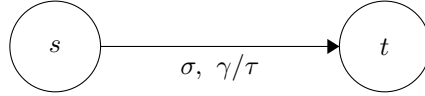


FIGURA 4. Tranziție într-un automat push-down

Observație 3.6. Pentru a simplifica notațiile, vom utiliza litere mici și cifre ca alfabet de input și majuscule pentru alfabetul stivei.

Observație 3.7. Simbolul Z_0 (uneori notat \$), se află mereu în stivă la început. Mereu va trebui să citim Z_0 de pe stivă la început și nu avem voie să nu îl punem la loc, decât când am finalizat de parcurs cuvântul.

Observație 3.8 (Instantanee). Pentru a descrie o etapă din parcurgerea unui cuvânt de către automat vom avea nevoie de un tuplu din $Q \times \Sigma^* \times \Gamma^*$, astfel memorând starea la care ne aflăm, simbolurile din cuvânt care mai trebuie consumate și configurația curentă a stivei.

Definiție 3.9 (Acceptarea cuvintelor). Automatele push-down au trei tipuri de acceptare:

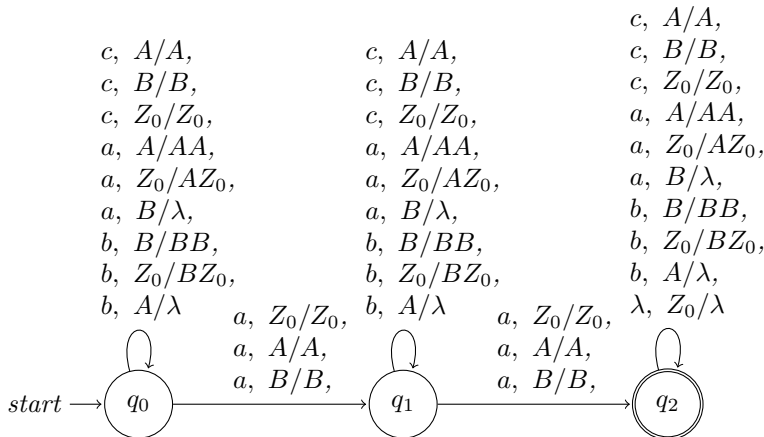
- (1) Prin stări finale
 $T(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q_f, \lambda, \alpha), q_f \in F\}$
- (2) Prin stiva vidă
 $N(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q, \lambda, \lambda)\}$
- (3) Prin stări finale și stiva vidă
 $L(A) = \{w \mid (q_0, w, Z_0) \vdash^* (q_f, \lambda, \lambda), q_f \in F\}$

Teoremă 3.10. Fie $\mathcal{T}_{PDA}, \mathcal{N}_{PDA}$ și $\mathcal{L}_{PDA}$ familiile limbajelor acceptate de automate pushdown cu stări finale, stiva vidă, respectiv stiva vidă și stări finale. Atunci:

$$\mathcal{N}_{PDA} = \mathcal{L}_{PDA} = \mathcal{T}_{PDA}$$

Exemplu 3.11. Automat push-down cu acceptare cu stări finale și stivă vidă pentru limbajul:

$$L = \{w \in \{a, b, c\}^* \mid |w|_a = |w|_b + 2\}$$



Definiție 3.12. Un automat push-down este determinist (DPDA) dacă și numai dacă respectă următoarele proprietăți Pentru orice $q \in Q, a \in \Sigma$ și $X \in \Sigma$:

- (1) $\delta(q, a, X)$ are cel mult un element
- (2) Dacă $\exists \delta(q, a, X)$, atunci $\nexists \delta(q, \lambda, X)$.

Teoremă 3.13. Fie $\mathcal{T}_{DPDA}$, $\mathcal{N}_{DPDA}$ și $\mathcal{L}_{DPDA}$ familiile limbajelor acceptate de automate pushdown deterministe cu stări finale, stiva vidă, respectiv stiva vidă și stări finale. Atunci:

$$\mathcal{N}_{DPDA} = \mathcal{L}_{DPDA} \subset \mathcal{T}_{DPDA}$$

4. INDICAȚII DE IMPLEMENTARE

Observație 4.1. In această secțiune nu urmărim o implementare completă, ci doar ideile principale care trebuie transpuse în cod pentru fiecare cerință. Este recomandat ca reprezentarea internă a automatelor să fie cât mai uniformă, pentru a putea refolosi funcții auxiliare.

4.1. Sugestie de reprezentare. O alegere convenabilă este să reprezentăm automatul prin mai multe variabile simple: mulțimea de stări, alfabetul, tranzițiile, starea inițială și mulțimea stărilor finale.

Pentru un NFA sau λ -NFA, putem folosi:

- mulțimea de stări: `set[str]`
- alfabetul: `set[str]`
- mulțimea stărilor finale: `set[str]`
- tranzițiile:

$$\text{dict[str, dict[str, set[str]]]}$$

Astfel, pentru o stare q și un simbol a , accesăm destinațiile prin:

$$\text{transitions}[q][a]$$

dacă acea tranziție există.

Pentru PDA, deoarece tranziția depinde și de simbolul din vârful stivei, putem folosi:

$$\text{dict[str, dict[tuple[str, str], list[tuple[str, str]]]]$$

unde:

- cheia exterioară este starea curentă;
- cheia interioară este perechea (`simbol_intrare`, `simbol_stivă`);
- valoarea este lista tranzițiilor posibile, fiecare de forma (`stare_următoare`, `înlocuire_stivă`).

Observație 4.2. Este util să fixăm de la început o convenție pentru tranzițiile vide, de exemplu "`lambda`" sau "`eps`", și să o folosim peste tot în program.

4.2. 1. Transformare λ -NFA $\rightarrow$ DFA minim. Implementarea poate fi împărțită în trei pași mari.

Pasul 1: Calculul λ -inchiderii. Pentru fiecare stare q , trebuie calculată mulțimea tuturor stărilor accesibile din q folosind doar tranziții λ .

O abordare simplă este DFS sau BFS pornind din starea curentă.

```
from collections import deque
```

```
def lambda_closure(state, transitions, lambda_symbol="lambda"):
    closure = {state}
    queue = deque([state])

    while queue:
        current = queue.popleft()
        next_states = transitions.get(current, {}).get(lambda_symbol, set())

        for nxt in next_states:
            if nxt not in closure:
                closure.add(nxt)
                queue.append(nxt)

    return closure
```

Observație 4.3. *In practică, este mai bine să calculăm λ -inchiderea pentru fiecare stare o singură dată, nu să o recalculăm de fiecare dată în subset construction.*

*Pasul 2: Subset construction. Stările DFA-ului vor fi submulțimi de stări din automatul inițial. Pentru a le putea folosi drept chei în dicționare, este convenabil să le reprezentăm prin **frozenset**.*

Ideea de bază:

- starea inițială a DFA-ului este λ -inchiderea stării inițiale din λ -NFA;
- pentru o stare-submulțime R și un simbol $a \neq \lambda$, se calculează toate stările accesibile cu a din oricare stare din R , apoi se aplică din nou λ -inchiderea;
- o stare a DFA-ului este finală dacă conține cel puțin o stare finală din automatul inițial.

```
def move(states, symbol, transitions):
    result = set()

    for state in states:
        result |= transitions.get(state, {}).get(symbol, set())

    return result
```

Observație 4.4. *Este important să ignorăm simbolul λ atunci când construim alfabetul DFA-ului.*

Pasul 3: Minimizarea. În implementare, pentru dimensiuni mici și medii, algoritmul lui Moore este de obicei mai ușor de scris și de verificat decât Hopcroft.

Pașii esențiali sunt:

- eliminarea stărilor inaccesibile;
- pornirea de la partiția $\{F, Q \setminus F\}$;
- separarea repetată a claselor după semnătura tranzițiilor;
- oprirea când partiția nu se mai schimbă.

O semnătură utilă pentru o stare este tuplul format din indicele clasei în care ajunge pentru fiecare simbol din alfabet.

Observație 4.5. *Chiar dacă teoretic Hopcroft este mai eficient, în contextul temei este mai importantă corectitudinea implementării și puteți alege Moore.*

4.3. 2. Transformare λ -NFA $\rightarrow$ Expresie Regulată. *Aici este util să lucrăm cu un automat finit extins, adică pe muchii să avem etichete de tip expresie regulată, nu doar simboluri.*

Idei principale.

- Unim tranzițiile paralele dintre aceleași două stări folosind reuniunea.
- Dacă este nevoie, adăugăm o stare inițială nouă și o stare finală nouă.
- Eliminăm pe rând stările intermediare.

Dacă eliminăm starea q , dar există muchii

$$p \xrightarrow{R_1} q, \quad q \xrightarrow{R_2} q, \quad q \xrightarrow{R_3} r,$$

atunci între p și r adăugăm sau actualizăm eticheta:

$$R_1 \cdot (R_2)^* \cdot R_3.$$

Observație 4.6. *Este bine să existe funcții separate pentru reuniune, concatenare și stelare, deoarece ele pot face automat mici simplificări:*

$$\emptyset + R = R, \quad \lambda \cdot R = R, \quad (\emptyset)^* = \lambda.$$

Mic exemplu de funcție utilă:

```
def regex_union(r1, r2):
    if r1 == "":
        return r2
    if r2 == "":
        return r1
```

```

if r1 == r2:
    return r1
return f"({r1}+{r2})"

```

Observație 4.7. *In practică, dificultatea principală nu este algoritmul în sine, ci păstrarea expresiilor suficient de clare, cu paranteze puse corect.*

4.4. 3. Simulator de Automat Push-Down (PDA). *Pentru PDA este cel mai natural să lucrăm cu configurații de forma:*

(stare curentă, poziție în input, conținutul stivei)

Deoarece automatul poate fi nedeterminist, nu urmărim o singură execuție, ci o mulțime de configurații posibile.

Ideea de implementare. O soluție potrivită este BFS:

- pornim din configurația inițială;
- extragem pe rând o configurație;
- generăm toate configurațiile următoare posibile;
- acceptăm imediat când găsim o configurație care satisface condiția de acceptare.

O configurație poate fi memorată simplu ca un tuplu:

(state, index, stack)

unde state este starea curentă, index este poziția în șirul de intrare, iar stack este stiva curentă.

De exemplu:

config = ("q0", 0, ("Z0",))

Observație 4.8. *Pentru a evita bucle infinite produse de tranziții λ , este important să memorăm configurațiile deja vizitate.*

La fiecare tranziție:

- verificăm simbolul din input, dacă tranziția nu este cu λ ;
- verificăm simbolul din vârful stivei;
- facem pop pentru simbolul cerut;
- apoi adăugăm pe stivă șirul nou, de regulă în ordine inversă dacă folosim capătul listei sau al tuplului drept vârf al stivei.

Condiția de acceptare trebuie implementată exact conform cerinței:

- prin stare finală;
- sau prin stivă vidă;
- sau prin ambele.

4.5. 4. Bonus: Expresie Regulată $\rightarrow \lambda$ -NFA. *Această cerință are două etape importante: parsarea expresiei regulate și construirea automatului echivalent.*

Pasul 1: Parsarea expresiei. Pentru a aplica mai ușor algoritmul de construcție, este util să transformăm expresia regulată din forma obișnuită într-o formă mai ușor de prelucrat.

Mai întâi, concatenarea trebuie făcută explicită. În scrierea obișnuită, concatenarea este de multe ori subînțeleasă. De exemplu, expresia

$ab^*|c$

se rescrie mai întâi sub forma

$a.b^*|c$

unde simbolul . marchează explicit concatenarea.

După aceea, expresia se transformă din formă infixată în formă postfixată folosind algoritmul Shunting Yard. Pentru exemplul de mai sus, forma postfixată este:

$ab * .c|$

Observație 4.9. *Este bine să tratăm separat cele două etape:*

- mai întâi inserăm operatorul de concatenare;

- apoi aplicăm algoritmul Shunting Yard.

Observație 4.10. Concatenarea este adesea partea care produce cele mai multe erori, pentru că în expresia inițială ea este implicită.

Pasul 2: Construcția Thompson. După obținerea formei postfixate, expresia se parcurge de la stânga la dreapta, folosind o stivă de fragmente de automat.

Ideea este următoarea:

- pentru fiecare simbol obișnuit se construiește un automat elementar, cu o stare de start, o stare finală și o singură tranziție între ele;
- la concatenare se scot din stivă ultimele două fragmente și se leagă finalul primului de începutul celui de-al doilea;
- la reuniune se construiește un nou fragment, cu stare inițială nouă și stare finală nouă, legate prin tranziții λ de cele două automate extrase din stivă;
- la stea se adaugă o stare inițială nouă și o stare finală nouă, astfel încât automatul să poată repeta de oricâte ori fragmentul existent sau să accepte direct cuvântul vid.

La final, în stivă trebuie să rămână un singur fragment, care reprezintă automatul λ -NFA c .

Observație 4.11. Este util ca fiecare fragment să fie memorat prin:

- starea de start;
- starea finală;
- mulțimea de tranziții.

4.6. Observații finale.

- Este recomandată separarea clară între:
 - parsarea inputului;
 - logica algoritmică;
 - afișarea outputului.

Observație 4.12. În toate cerințele, o implementare corectă și clară este de preferat uneia foarte compacte, dar greu de urmărit.